DSA with Python

Day 1

Mastering Data Structures, Algorithms, Time Complexity, and Core Python Array Operations to kickstart your coding journey.

🎯 Today's Goal

Core Concepts

Understand exactly what **Data Structures** are and how **Algorithms** function to solve computational problems step-by-step.

Efficiency

Learn what **Time Complexity** means and why it's critical for writing fast, scalable, and professional code.

Python Action

Master basic array (list) operations in Python and apply them immediately to solve your first set of DSA problems.

1️⃣ What is a Data Structure?

A Data Structure is a systematic way to store and organize data in a computer so that it can be used efficiently.

🧠 Real-Life Example

Imagine a school classroom. Students are arranged systematically in a line of benches:

```
Rahul | Aman | Sita | Priya
```

This physical arrangement is exactly like an **Array** (or List). It allows you to easily check, replace, or count students. The arrangement itself is the data structure.

2️⃣ What is an Algorithm?

An Algorithm is a finite, step-by-step method to solve a specific problem.

🧠 Real-Life Example: Making Maggi

If the problem is hunger, the steps are:

1. Boil water in a pot
2. Add Maggi noodles
3. Add the masala
4. Cook for exactly 2 minutes

These sequence of steps form the algorithm. In programming, the cycle is always:
Problem → Steps → Solution.

3️⃣ What is Time Complexity?

Time Complexity evaluates how fast an algorithm runs. Imagine trying to find a specific book in a massive library.

Method 1 — Check Every Book

You start at the very first shelf and read the spine of Book 1, then Book 2, then Book 3, and so on until you find your target.

Highly Inefficient (Slow)
This takes a long time, especially if the library is huge.

Method 2 — Use Library Catalog

You look up the exact index of the book in the computer system, which points you directly to the correct aisle and shelf.

⚡ Highly Efficient (Fast)
Directly accessing the item takes almost zero time. Time complexity mathematically measures this efficiency difference!

Common Time Complexities

Complexity Notation	Meaning	Example Scenario
$O(1)$	Instant / Constant Time	Directly looking up an item if you know its exact location (index).
$O(n)$	Linear Time (Depends on data size)	Checking 10 numbers is fast; checking 1 million numbers takes proportionally longer.
$O(n^2)$	Quadratic Time (Very slow)	Comparing every item in a list with every other item in the same list.

4 Python Arrays (Lists)

Lists as Arrays: In Python, the concept of arrays is implemented using Lists. They can store multiple values in a single variable.

```
numbers = [10, 20, 30, 40, 50]
```

Accessing Elements: You access items using their index. Remember, indexes always start from **0**.

```
print(numbers[0]) # Output: 10
```

```
Index: 0   1   2   3
Value: 10  20  30  40
```

Traversing a List: You can loop through each item one by one easily using a `for` loop.

```
for num in numbers:
    print(num)
```

5️⃣ Problem 1: Find Maximum Number

The Logic & Code

Goal: Find the largest number in `[4, 8, 1, 9, 3]`.

Expected Answer: 9.

> 🧠 **Real-Life Concept:** Imagine a teacher finding the tallest student. The teacher checks the first height, then the second. If the second is taller, that becomes the new "tallest". They repeat this until the end.

```
numbers = [4, 8, 1, 9, 3]
max_num = numbers[0]

for num in numbers:
    if num > max_num:
        max_num = num

print("Maximum:", max_num) # Output: Maximum: 9
```

6️⃣ Problem 2: Find Minimum Number

Target List:

```
[4, 8, 1, 9, 3]
```

1

Final Minimum Output

🧠 Performing a Dry Run

Tracking variables step-by-step is called a dry run. Let's trace our logic manually.

Start: `min_num = 4` (Assume first is smallest)

Check 8: Is 8 < 4? **No.**

Check 1: Is 1 < 4? **Yes!** → **update min_num = 1**

Check 9: Is 9 < 1? **No.**

Check 3: Is 3 < 1? **No.**

🧪 Practice Questions

Time to write some code! Try solving these three challenges using the concepts you learned today.

Q1: Sum Elements

Find the sum of all elements in the given list.

```
[5, 10, 15, 20]
```

Expected: 50

Q2: Count Elements

Write logic to count the total number of items in the list (without using Python's `len` function).

```
[2, 4, 6, 8, 10]
```

Q3: Reverse Order

Print the elements of the list in complete reverse order.

```
[1, 2, 3, 4, 5]
```

Output: 5 4 3 2 1

⏱ Your 2-Hour Plan Today

Fo ne to solidify today's learnings.

0 - 30 min

Review Concepts: Re-read definitions of Data Structures, Algorithms, and Time Complexity.

30 - 60 min

Code & Dry Run: Type out the Min/Max code yourself. Perform manual dry runs on paper.

60 - 90 min

Practice Q1 & Q2: Spend focused time solving the Sum and Count list problems.

90 - 120 min

Practice Q3 & Review: Attempt the reverse order problem and review all your code.

💡 Tomorrow (Day 2)

Get ready to dive deeper!

Tomorrow, we will build upon these foundations and tackle more complex concepts:

- Arrays deeply under the hood
- Advanced array traversal techniques
- Solving Sum and Average calculation patterns
- ⭐ **Important interview questions**

Rest well and see you tomorrow!

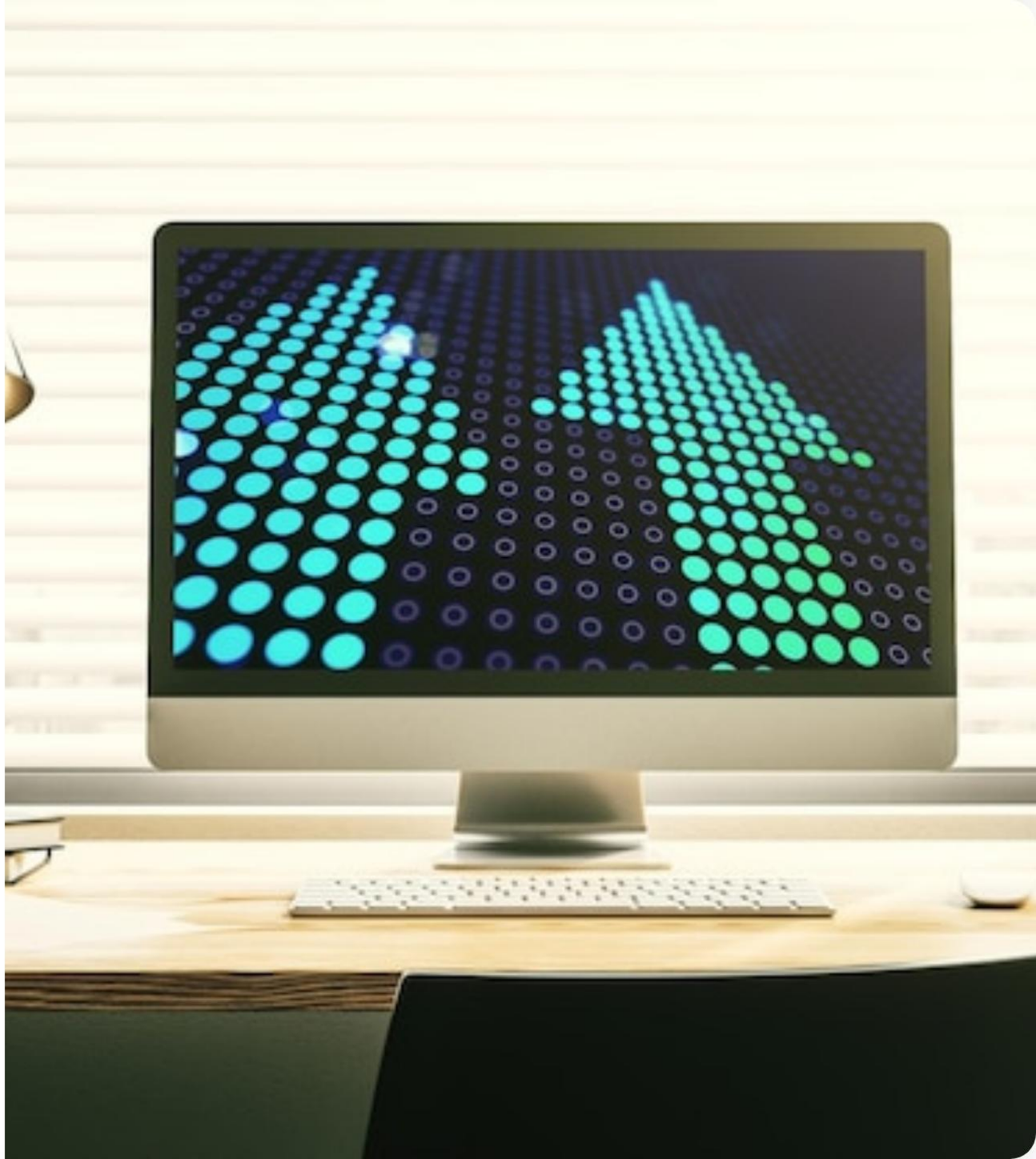

Image Sources

https://computercomforts.com/wp-content/uploads/basic-table-image-04.jpg

Source: computercomforts.com

https://png.pngtree.com/thumb_back/fh260/background/20251204/pngtree-steaming-noodles-being-cooked-in-a-pot-on-modern-stovetop-bright-image_20722623.webp

Source: pngtree.com

https://cdn.vectorstock.com/i/1000v/37/81/child-measuring-height-with-teacher-vector-28593781.jpg

Source: www.vectorstock.com

https://img.freepik.com/premium-photo/creative-abstract-upward-arrows-sketch-modern-laptop-monitor-target-goal-concept-3d-rendering_258654-11029.jpg

Source: www.freepik.com